Eltex

ОТЧЁТ

по практической работе № 3

Раздел 1.

Студент: Киселёв Е. С.

Преподаватель: Межулин С. А.

Подгруппа: 1

Номер: 21

Новосибирск — 2026

Цель работы

Изучить организацию процессов и потоков выполнения в операционной системе Linux, освоить просмотр и анализ состояния процессов средствами ps и top, интерпретацию показателей использования памяти и идентификаторов пользователей. Приобрести практические навыки отправки и перехвата сигналов, управления заданиями в интерактивной оболочке, изменения и проверки значения NICE, а также исследования жизненного цикла дочерних и зомби-процессов. Закрепить навыки документирования команд, результатов и наблюдений.

Задание

1. Войдите под пользователем user1 из практики 2 (su - user1).

2. Подсчитайте количество процессов, имеющих несколько потоков выполнения.

3. Запустите top и настройте вывод: отсортируйте процессы по убыванию VIRT, RES и SHR с фиксацией результатов и объяснением показателей; удалите поля VIRT, RES, SHR, добавьте RUSER непосредственно после USER и объясните их различие.

4. В другом терминальном окне выполните passwd и оставьте команду в состоянии запроса текущего пароля.

5. В терминале с top выведите процессы, у которых реальный пользователь — вошедший в сеанс пользователь; найдите passwd; отправьте ему SIGTERM, SIGINT, SIGQUIT и SIGKILL по числовым идентификаторам (man 7 signal).

6. Выполните vim ~/file_task3.txt и нажмите Ctrl+Z.

7. Выполните sleep 600, нажмите Ctrl+Z и выполните jobs.

8. Сделайте последнее задание sleep 600 фоновым.

9. Измените NICE задания sleep 600 на 10.

10. Проверьте изменение NICE этого задания.

11. Сделайте задание vim ~/file_task3.txt активным и выйдите из редактора.

12. Отправьте SIGTERM заданию sleep 600 и выполните jobs.

13. Создайте с помощью trap перехватчик SIGINT и SIGQUIT с сообщением «Меня голыми руками не возьмёшь!» и отправьте сигналы самому себе.

14. Несколько раз выполните (sleep 1 & exec /bin/sleep 30) & для создания зомби-процессов; при необходимости увеличьте задержку 30 секунд.

15. Найдите командой ps зомби-процессы, выведите PID и PPID, попробуйте завершить их досрочно и опишите особенности примерно в 10 предложениях.

Ход выполнения работы

Пункт 1. Проверка учётной записи user1 и вход в её сеанс.

Выполненные действия и команды:

```
whoami
id user1
su - user1
whoami; pwd
```

Результат и пояснение. Подтверждены UID 1001, группы user1 и practice2, успешный вход и каталог /home/user1.

Результат соответствующего этапа показан на рисунке 1.

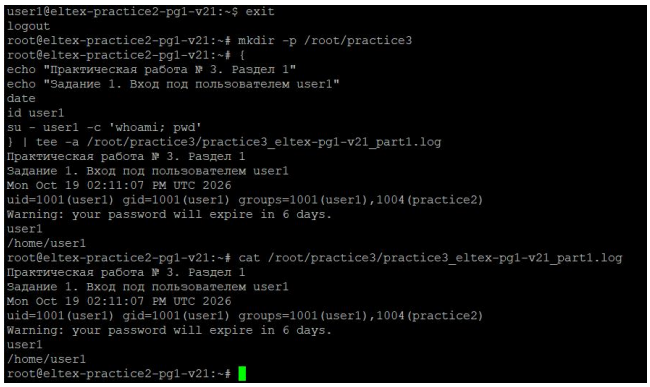


Рисунок 1 — Вход под user1 и проверка учётной записи

Результат соответствующего этапа показан на рисунке 2.

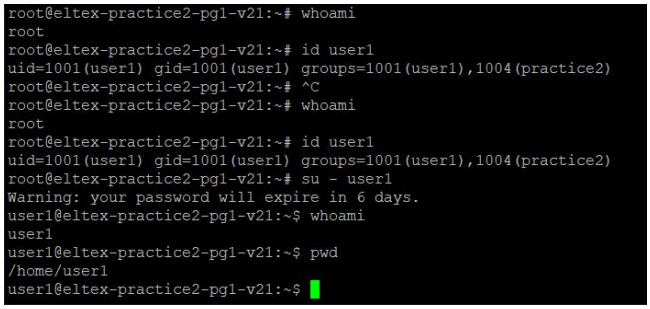


Рисунок 2 — Вход под user1 и проверка учётной записи

Пункт 2. Подсчёт процессов, имеющих более одного потока выполнения.

Выполненные действия и команды:

```
ps -e -o pid=,nlwp= | awk '$2 > 1 {count++} END {print "Количество многопоточных процессов:", count+0}'
```

Результат и пояснение. Получено 10 многопоточных процессов; NLWP означает число потоков процесса.

Результат соответствующего этапа показан на рисунке 3.

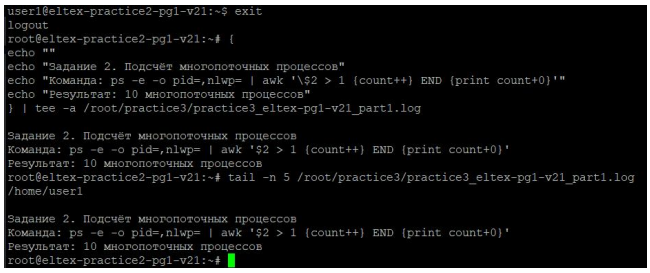


Рисунок 3 — Подсчёт многопоточных процессов

Результат соответствующего этапа показан на рисунке 4.

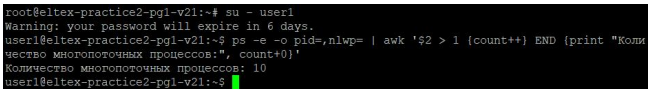


Рисунок 4 — Подсчёт многопоточных процессов

Пункт 3. Настройка полей и сортировки в интерактивной утилите top.

Выполненные действия и команды:

```
top
f  # управление полями
s  # выбор поля сортировки
q  # возврат к списку процессов
```

Результат и пояснение. Выполнена сортировка по убыванию VIRT, RES, SHR; после удаления этих полей RUSER помещено вслед за USER. VIRT — размер виртуального адресного пространства; RES — резидентная память в ОЗУ; SHR — часть резидентной памяти, потенциально совместно используемая. USER показывает эффективного пользователя (EUID), RUSER — реального (RUID).

Результат соответствующего этапа показан на рисунке 5.

```
user1@eltex-practice2-pg1-v21:~$ exit
logout
root@eltex-practice2-pg1-v21:~# cat >> /root/practice3/practice3_eltex-pg1-v21_part1.log <<'E
OF'

Задание 3. Настройка отображения процессов в top

1. Запущена команда top под пользователем user1.
2. Выполнена сортировка процессов по убыванию VIRT.
3. Выполнена сортировка процессов по убыванию RES.
4. Выполнена сортировка процессов по убыванию SHR.
5. Поля VIRT, RES, SHR отключены.
6. Поле RUSER добавлено непосредственно после USER.

VIRT — общий размер виртуального адресного пространства процесса.
RES — объем физической оперативной памяти, занимаемой процессом.
SHR — объем потенциально разделяемой памяти процесса.

USER — эффективный пользователь процесса (EUID).
RUSER — реальный пользователь процесса (RUID).

Результаты настройки и сортировки зафиксированы скриншотами.
EOF
root@eltex-practice2-pg1-v21:~# tail -n 20 /root/practice3/practice3_eltex-pg1-v21_part1.log
Команда: ps -e -o pid=,nlwp= | awk '$2 > 1 {count++} END {print count+0}'
Результат: 10 многопоточных процессов

Задание 3. Настройка отображения процессов в top

1. Запущена команда top под пользователем user1.
2. Выполнена сортировка процессов по убыванию VIRT.
3. Выполнена сортировка процессов по убыванию RES.
4. Выполнена сортировка процессов по убыванию SHR.
5. Поля VIRT, RES, SHR отключены.
6. Поле RUSER добавлено непосредственно после USER.

VIRT — общий размер виртуального адресного пространства процесса.
RES — объем физической оперативной памяти, занимаемой процессом.
SHR — объем потенциально разделяемой памяти процесса.

USER — эффективный пользователь процесса (EUID).
RUSER — реальный пользователь процесса (RUID).

Результаты настройки и сортировки зафиксированы скриншотами.
root@eltex-practice2-pg1-v21:~#
```

Рисунок 5 — Настройка и сортировка процессов в top

Результат соответствующего этапа показан на рисунке 6.

```
top - 14:25:59 up 21:04,  3 users,  load average: 0.00, 0.00, 0.00
Tasks: 113 total,   1 running, 112 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   3915.9 total,   2959.8 free,    433.1 used,    753.0 buff/cache
MiB Swap:   3185.0 total,   3185.0 free,      0.0 used.   3482.8 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    895 root      20   0  634376  45380  39524 S   0.0   1.1   0:02.58 fwupd
    353 root      rt   0  289124  27460   8768 S   0.0   0.7   0:04.29 multipathd
    709 root      20   0  109700  22944  13492 S   0.0   0.6   0:00.11 unattended-upgr
    694 root      20   0  468988  13596  11368 S   0.0   0.3   0:00.80 udisksd
    310 root      19  -1   34084  13416  12256 S   0.0   0.3   0:00.41 systemd-journal
      1 root      20   0   22216  13352   9556 S   0.0   0.3   0:02.13 systemd
    552 systemd+  20   0   21480  13084  10860 S   0.0   0.3   0:00.17 systemd-resolve
    750 root      20   0  318380  12860  10788 S   0.0   0.3   0:00.08 ModemManager
   2846 root      20   0   20292  11488   9404 S   0.0   0.3   0:00.06 systemd
   4120 root      20   0   14984  10680   8712 S   0.0   0.3   0:00.03 sshd
   4373 root      20   0   14976  10668   8700 S   0.0   0.3   0:00.03 sshd
   3489 root      20   0   14980  10608   8636 S   0.0   0.3   0:00.02 sshd
    685 polkitd   20   0  383720  10036   7600 S   0.0   0.3   0:01.04 polkitd
    540 systemd+  20   0   19020   9504   8336 S   0.0   0.2   0:00.70 systemd-network
    902 root      20   0  313972   9060   7620 S   0.0   0.2   0:00.01 upowerd
    691 root      20   0   18196   8964   7884 S   0.0   0.2   0:00.12 systemd-logind
   2840 root      20   0   12028   8272   7124 S   0.0   0.2   0:00.00 sshd
    369 root      20   0   29228   7952   4996 S   0.0   0.2   0:00.18 systemd-udevd
    553 systemd+  20   0   91036   7840   6872 S   0.0   0.2   0:00.19 systemd-timesyn
    708 syslog    20   0  222516   6328   4624 S   0.0   0.2   0:00.08 rsyslogd
   4577 user1     20   0   12108   6140   3812 R   0.0   0.2   0:00.01 top
    680 message+  20   0    9916   5632   4692 S   0.0   0.1   0:00.18 dbus-daemon
   3546 root      20   0    8652   5620   3908 S   0.0   0.1   0:00.02 bash
   4472 root      20   0    8652   5608   3920 S   0.0   0.1   0:00.00 bash
   4177 root      20   0    8652   5580   3880 S   0.0   0.1   0:00.02 bash
   4558 user1     20   0    8648   5576   3864 S   0.0   0.1   0:00.01 bash
   4018 root      20   0    9308   5008   4576 S   0.0   0.1   0:00.00 dbus-daemon
   4557 root      20   0    9412   4680   4208 S   0.0   0.1   0:00.00 su
    686 root      20   0   80716   4276   3976 S   0.0   0.1   0:29.19 qemu-ga
   2847 root      20   0   21168   3616   1908 S   0.0   0.1   0:00.00 (sd-pam)
    679 root      20   0    6828   2964   2724 S   0.0   0.1   0:00.08 cron
    706 root      20   0    6108   2036   1900 S   0.0   0.1   0:00.00 agetty
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.01 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_g
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_p
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
     12 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_pe
     13 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
     14 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
     15 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
     16 root      20   0       0      0      0 S   0.0   0.0   0:00.02 ksoftirqd/0
```

Рисунок 6 — Настройка и сортировка процессов в top

Результат соответствующего этапа показан на рисунке 7.

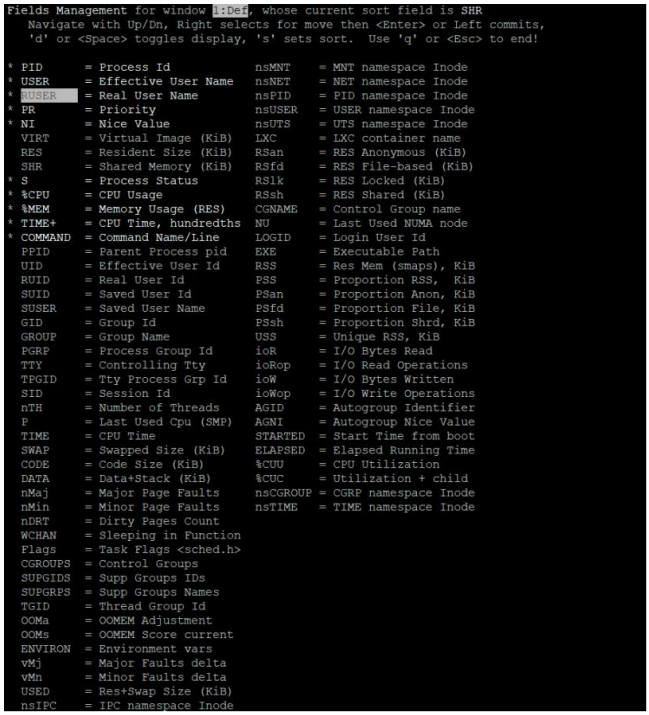


Рисунок 7 — Настройка и сортировка процессов в top

Результат соответствующего этапа показан на рисунке 8.

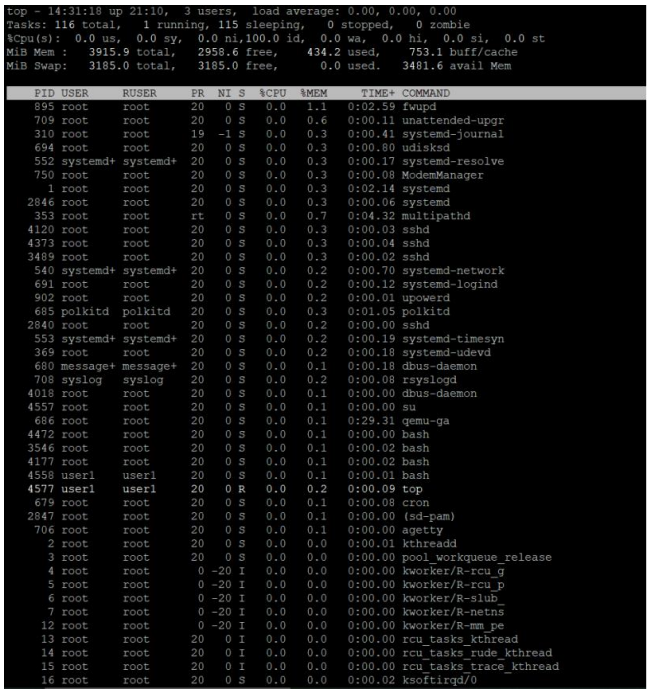


Рисунок 8 — Настройка и сортировка процессов в top

Результат соответствующего этапа показан на рисунке 9.

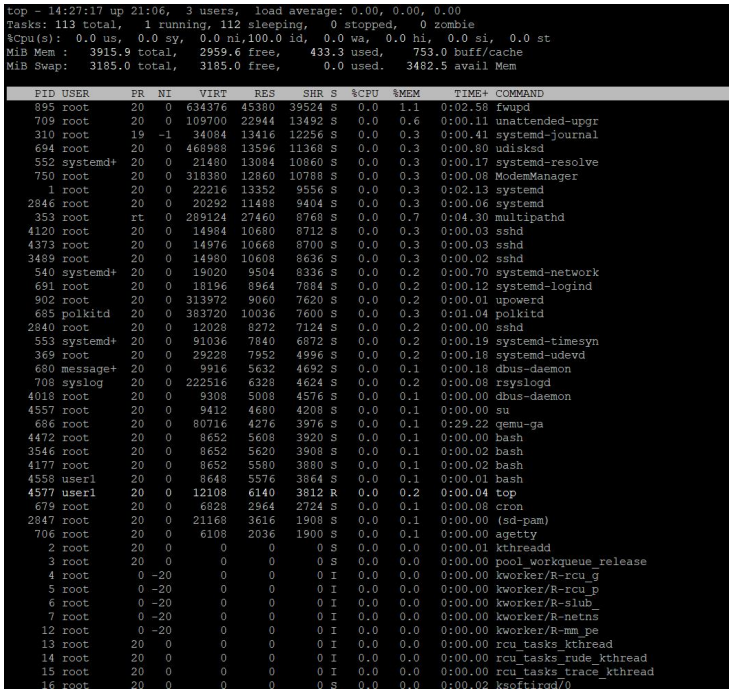


Рисунок 9 — Настройка и сортировка процессов в top

Результат соответствующего этапа показан на рисунке 10.

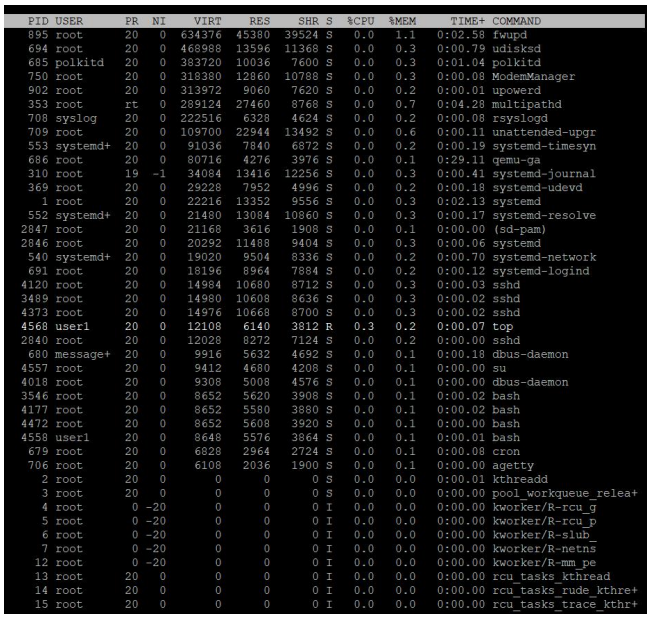


Рисунок 10 — Настройка и сортировка процессов в top

Результат соответствующего этапа показан на рисунке 11.

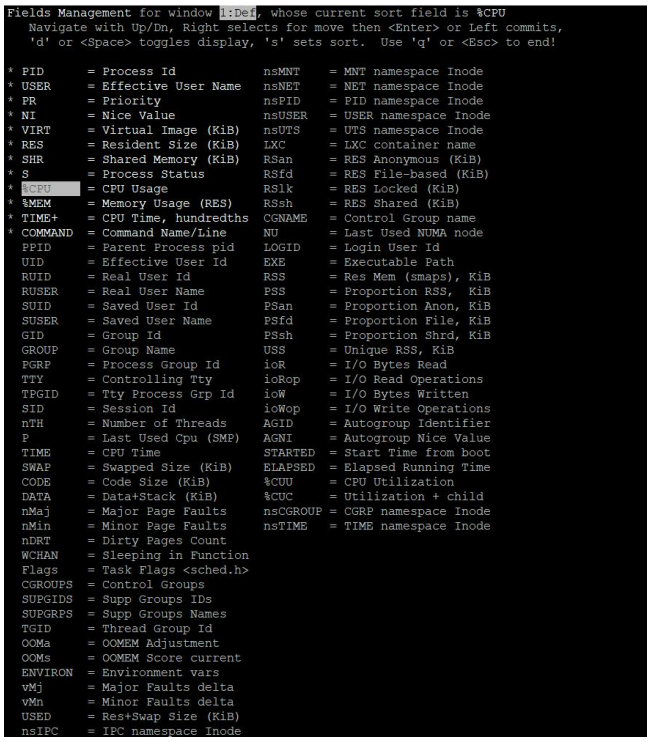


Рисунок 11 — Настройка и сортировка процессов в top

Пункт 4. Запуск passwd в отдельном терминале.

Выполненные действия и команды:

```
passwd
```

Результат и пояснение. Процесс оставлен на приглашении Current password:; изменение пароля не выполнялось.

Результат соответствующего этапа показан на рисунке 12.

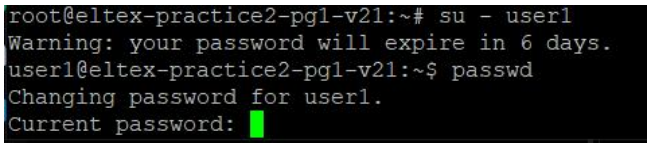


Рисунок 12 — Запрос текущего пароля командой passwd

Пункт 5. Фильтрация процессов по реальному пользователю и отправка сигналов passwd.

Выполненные действия и команды:

```
ps -eo pid,ppid,user,ruser,stat,comm | grep passwd
kill -15 4685
kill -2 4685
```

```
kill -3 4685
kill -9 4685
```

Результат и пояснение. У passwd PID 4685, PPID 4675, USER=root и RUSER=user1. SIGTERM (15), SIGINT (2), SIGQUIT (3) не завершили процесс; после SIGKILL (9) процесс исчез. Нулевой код kill подтверждает отправку сигнала, но сам по себе не гарантирует завершение процесса.

SIGTERM запрашивает штатное завершение, SIGINT соответствует прерыванию, SIGQUIT — завершению с возможным дампом, а SIGKILL не может быть перехвачен или проигнорирован исполняющимся процессом.

Результат соответствующего этапа показан на рисунке 13.

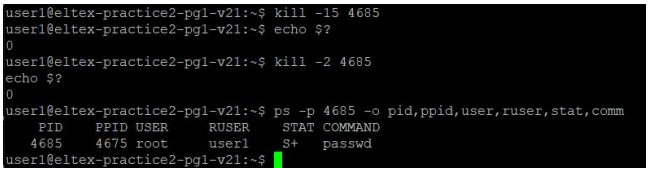


Рисунок 13 — Поиск passwd и проверка сигналов

Результат соответствующего этапа показан на рисунке 14.

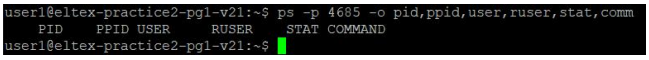


Рисунок 14 — Поиск passwd и проверка сигналов

Результат соответствующего этапа показан на рисунке 15.



Рисунок 15 — Поиск passwd и проверка сигналов

Результат соответствующего этапа показан на рисунке 16.

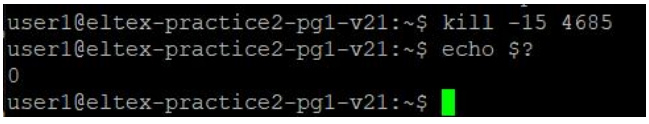


Рисунок 16 — Поиск passwd и проверка сигналов

Результат соответствующего этапа показан на рисунке 17.

```
root@eltex-practice2-pg1-v21:~# cat >> /root/practice3/practice3_eltex-pg1-v21_part1.log <<'EOF'

Задание 4. Запуск passwd
Под пользователем user1 выполнена команда passwd.
Процесс оставлен в состоянии запроса Current password:.

Задание 5. Управление процессом passwd
Команда поиска:
ps -eo pid,ppid,user,ruser,stat,comm | grep passwd

PID: 4685
PPID: 4675
USER: root
RUSER: user1

SIGTERM (15): отправлен, код 0, процесс продолжил работу.
SIGINT (2): отправлен, код 0, процесс продолжил работу.
SIGQUIT (3): отправлен, код 0, процесс продолжил работу.
SIGKILL (9): после отправки процесс отсутствует в списке ps.

Вывод: passwd имеет эффективного пользователя root
и реального пользователя user1. Первые три сигнала
не завершили процесс. После SIGKILL процесс исчез.
EOF
root@eltex-practice2-pg1-v21:~# tail -n 22 /root/practice3/practice3_eltex-pg1-v21_part1.log

Задание 4. Запуск passwd
Под пользователем user1 выполнена команда passwd.
Процесс оставлен в состоянии запроса Current password:.

Задание 5. Управление процессом passwd
Команда поиска:
ps -eo pid,ppid,user,ruser,stat,comm | grep passwd

PID: 4685
PPID: 4675
USER: root
RUSER: user1

SIGTERM (15): отправлен, код 0, процесс продолжил работу.
SIGINT (2): отправлен, код 0, процесс продолжил работу.
SIGQUIT (3): отправлен, код 0, процесс продолжил работу.
SIGKILL (9): после отправки процесс отсутствует в списке ps.

Вывод: passwd имеет эффективного пользователя root
и реального пользователя user1. Первые три сигнала
не завершили процесс. После SIGKILL процесс исчез.
root@eltex-practice2-pg1-v21:~#
```

Рисунок 17 — Поиск passwd и проверка сигналов

Результат соответствующего этапа показан на рисунке 18.

```
user1@eltex-practice2-pg1-v21:~$ ps -eo pid,ppid,user,ruser,stat,comm | grep pas
swd
   4685    4675 root     user1    S+   passwd
user1@eltex-practice2-pg1-v21:~$
```

Рисунок 18 — Поиск passwd и проверка сигналов

Результат соответствующего этапа показан на рисунке 19.

```
user1@eltex-practice2-pg1-v21:~$ ps -eo pid,ppid,user,ruser,stat,comm | grep pas
swd
   4685    4675 root     user1    S+   passwd
user1@eltex-practice2-pg1-v21:~$ top
top - 14:45:13 up 21:24,  4 users,  load average: 0.00, 0.00, 0.00
Tasks: 120 total,   1 running, 119 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   3915.9 total,   2946.9 free,    441.8 used,    757.3 buff/cache
MiB Swap:   3185.0 total,   3185.0 free,      0.0 used.   3474.0 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   4589 user1     20   0    8648   5624   3908 S   0.0   0.1   0:00.01 bash
   4675 user1     20   0    8648   5596   3884 S   0.0   0.1   0:00.01 bash
   4685 root      20   0    9176   3888   3540 S   0.0   0.1   0:00.00 passwd
   4694 user1     20   0   11944   6096   3808 R   0.0   0.2   0:00.00 top
```

Рисунок 19 — Поиск passwd и проверка сигналов

Результат соответствующего этапа показан на рисунке 20.

```
user1@eltex-practice2-pg1-v21:~$ ps -p 4685 -o pid,ppid,user,ruser,stat,comm
    PID    PPID USER     RUSER    STAT COMMAND
   4685    4675 root     user1    S+   passwd
user1@eltex-practice2-pg1-v21:~$
```

Рисунок 20 — Поиск passwd и проверка сигналов

Результат соответствующего этапа показан на рисунке 21.

```
user1@eltex-practice2-pg1-v21:~$ ps -p 4685 -o pid,ppid,user,ruser,stat,comm
    PID    PPID USER     RUSER    STAT COMMAND
   4685    4675 root     user1    S+   passwd
user1@eltex-practice2-pg1-v21:~$
```

Рисунок 21 — Поиск passwd и проверка сигналов

Результат соответствующего этапа показан на рисунке 22.

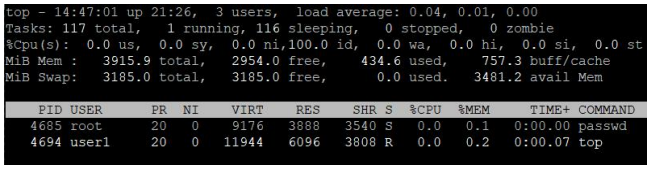


Рисунок 22 — Поиск passwd и проверка сигналов

Результат соответствующего этапа показан на рисунке 23.

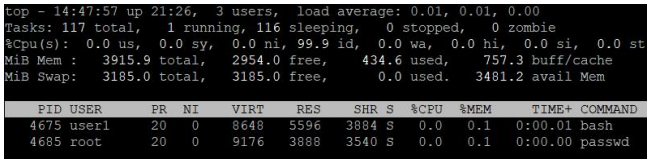


Рисунок 23 — Поиск passwd и проверка сигналов

Результат соответствующего этапа показан на рисунке 24.

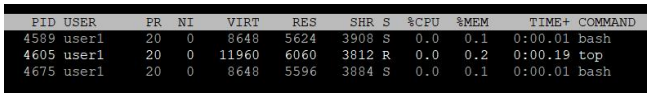


Рисунок 24 — Поиск passwd и проверка сигналов

Результат соответствующего этапа показан на рисунке 25.

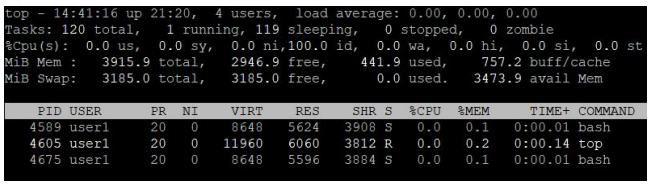


Рисунок 25 — Поиск passwd и проверка сигналов

Пункт 6. Приостановка Vim с помощью управления заданиями.

Выполненные действия и команды:

```
vim ~/file_task3.txt
Ctrl+Z
```

Результат и пояснение. Редактор приостановлен; оболочка сообщила [1]+ Stopped.

Результат соответствующего этапа показан на рисунке 26.

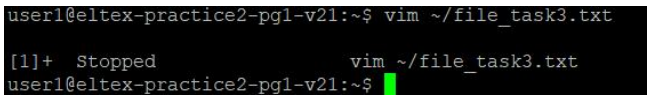


Рисунок 26 — Приостановка Vim

Пункт 7. Приостановка sleep и проверка списка заданий.

Выполненные действия и команды:

```
sleep 600
Ctrl+Z
jobs
```

Результат и пояснение. Оба задания vim и sleep 600 отображены в состоянии Stopped.

Результат соответствующего этапа показан на рисунке 27.

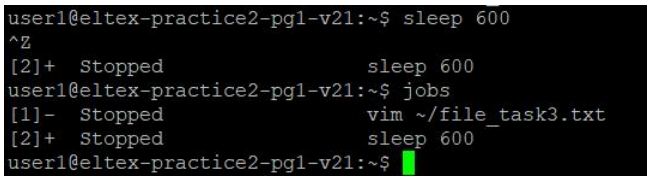


Рисунок 27 — Состояние заданий после Ctrl+Z

Пункт 8. Возобновление sleep в фоновом режиме.

Выполненные действия и команды:

```
bg %2
jobs
```

Результат и пояснение. Задание [2] sleep 600 перешло в Running, редактор vim остался Stopped.

Результат соответствующего этапа показан на рисунке 28.

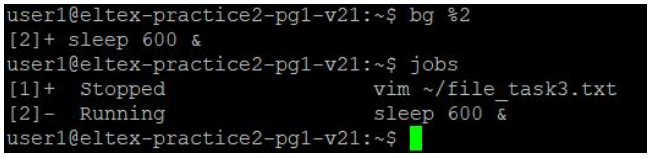


Рисунок 28 — Перевод sleep 600 в фоновый режим

Пункт 9. Изменение приоритета планирования.

Выполненные действия и команды:

```
renice -n 10 -p $(jobs -pr %2)
```

Результат и пояснение. Для PID 4749 значение NICE изменилось с 0 на 10. Положительное NICE означает меньший приоритет при обычном планировании.

Результат соответствующего этапа показан на рисунке 29.

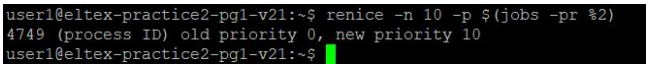


Рисунок 29 — Изменение NICE на 10

Пункт 10. Проверка нового значения NICE.

Выполненные действия и команды:

```
ps -p 4749 -o pid,comm,ni,stat
jobs
```

Результат и пояснение. Для sleep (PID 4749) показаны NI=10 и STAT=SN: S — ожидание, N — положительное NICE.

Результат соответствующего этапа показан на рисунке 30.

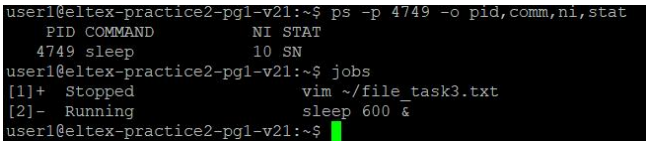


Рисунок 30 — Проверка NICE и состояния процесса

Пункт 11. Возврат Vim на передний план и выход.

Выполненные действия и команды:

```
fg %1
:qa!
jobs
```

Результат и пояснение. Редактор Vim возобновлён и закрыт; в списке заданий остался работающий sleep 600.

Результат соответствующего этапа показан на рисунке 31.

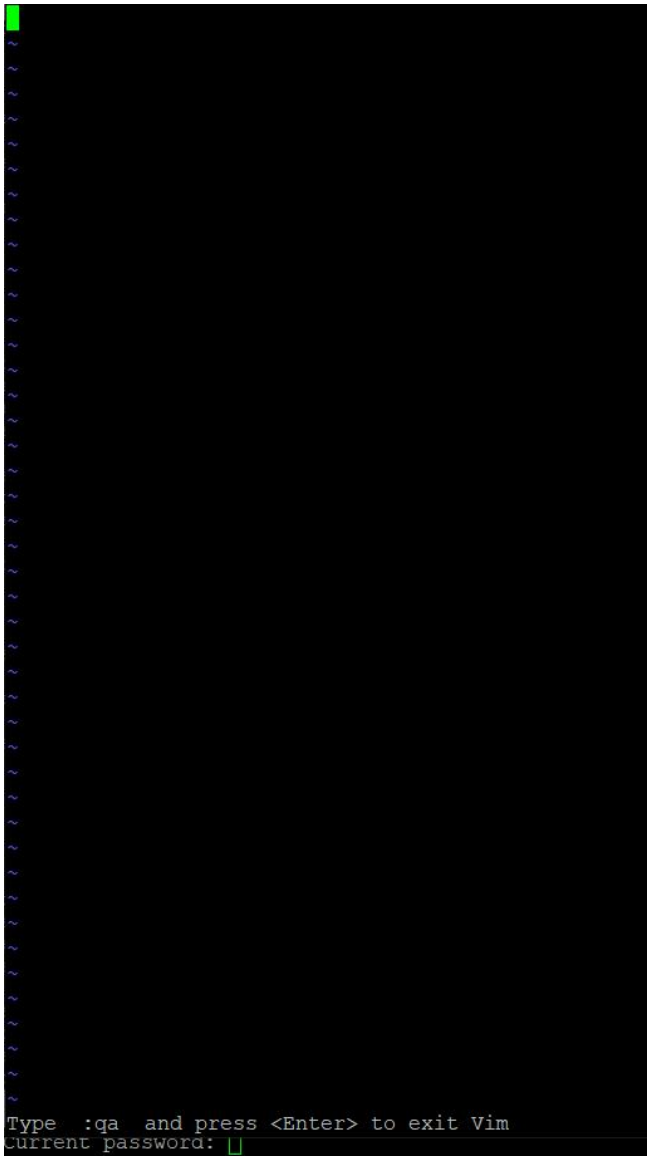


Рисунок 31 — Возобновление и закрытие Vim

Результат соответствующего этапа показан на рисунке 32.

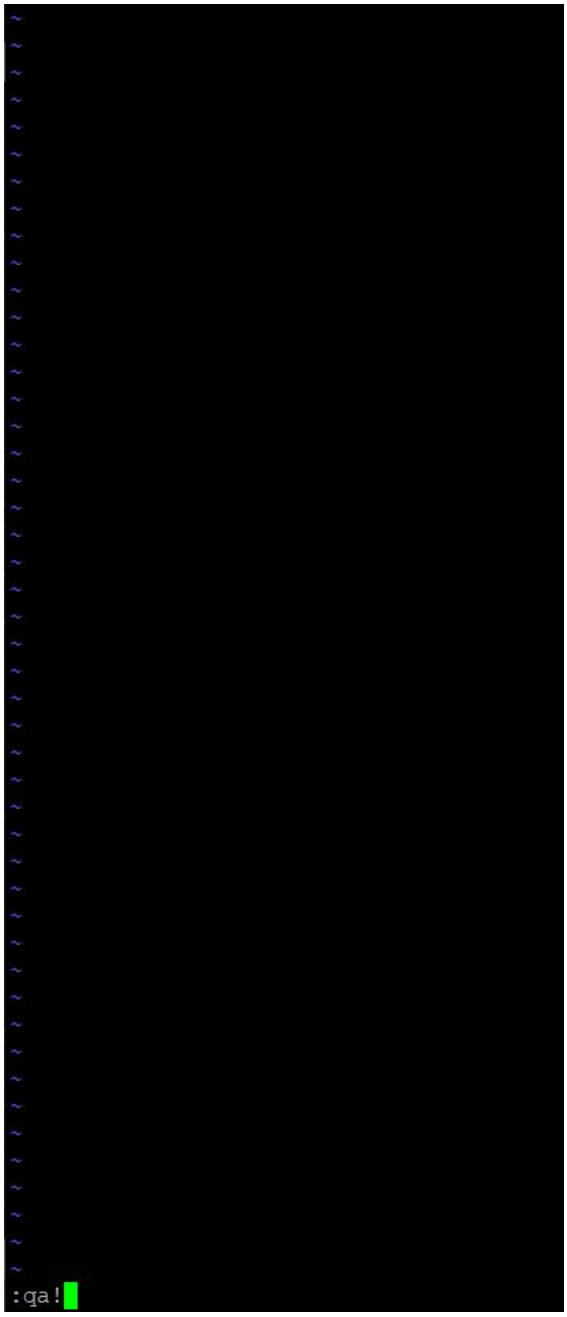


Рисунок 32 — Возобновление и закрытие Vim

Результат соответствующего этапа показан на рисунке 33.

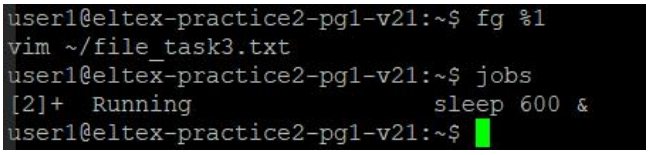


Рисунок 33 — Возобновление и закрытие Vim

Пункт 12. Завершение фонового sleep.

Выполненные действия и команды:

```
kill -15 %2
jobs
```

Результат и пояснение. Выведено [2]+ Terminated sleep 600; активных заданий после проверки не осталось.

Результат соответствующего этапа показан на рисунке 34.

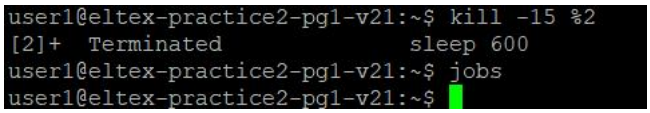


Рисунок 34 — Завершение sleep сигналом SIGTERM

Пункт 13. Настройка и проверка перехвата сигналов Bash.

Выполненные действия и команды:

```
trap 'echo "Меня голыми руками не возьмёшь!"' INT QUIT
trap -p INT QUIT
kill -2 $$
kill -3 $$
echo "Оболочка продолжает работать, PID: $$"
```

Результат и пояснение. Оба сигнала вызвали заданное сообщение; Bash продолжил работу с PID 4780.

Обработчик trap установлен в текущей оболочке: $$ раскрывается в её PID. После получения SIGINT и SIGQUIT вместо прекращения работы выполнена команда echo.

Результат соответствующего этапа показан на рисунке 35.

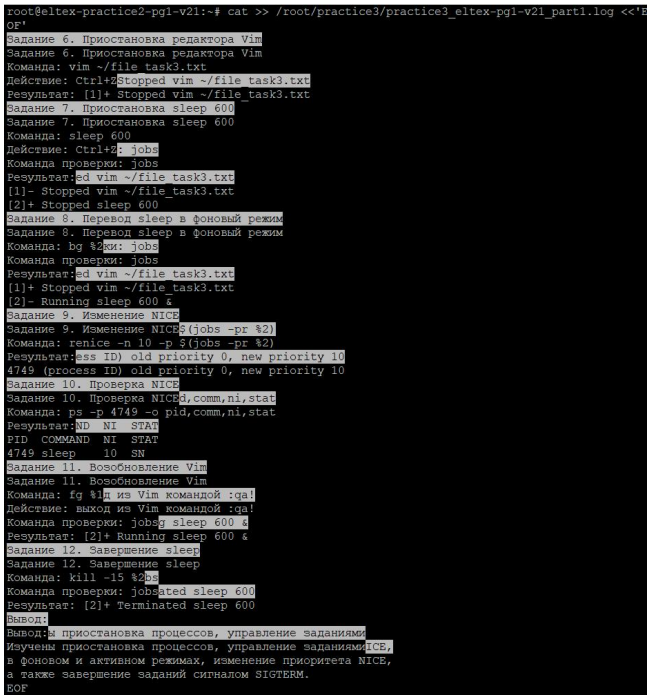


Рисунок 35 — Перехват SIGINT и SIGQUIT

Результат соответствующего этапа показан на рисунке 36.

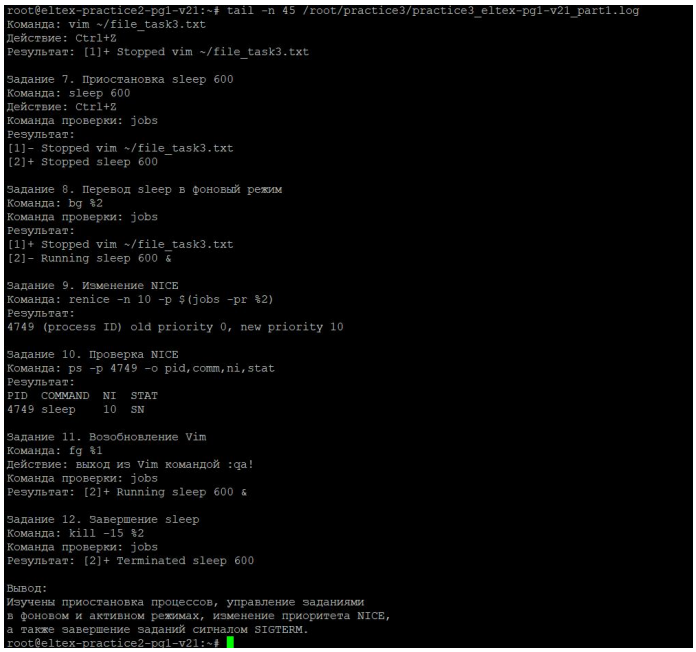


Рисунок 36 — Перехват SIGINT и SIGQUIT

Результат соответствующего этапа показан на рисунке 37.

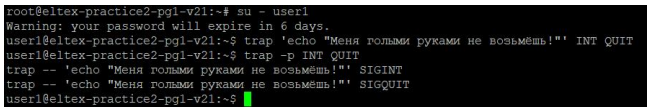


Рисунок 37 — Перехват SIGINT и SIGQUIT

Результат соответствующего этапа показан на рисунке 38.

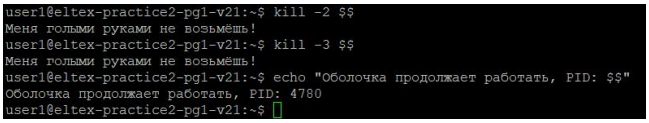


Рисунок 38 — Перехват SIGINT и SIGQUIT

Результат соответствующего этапа показан на рисунке 39.

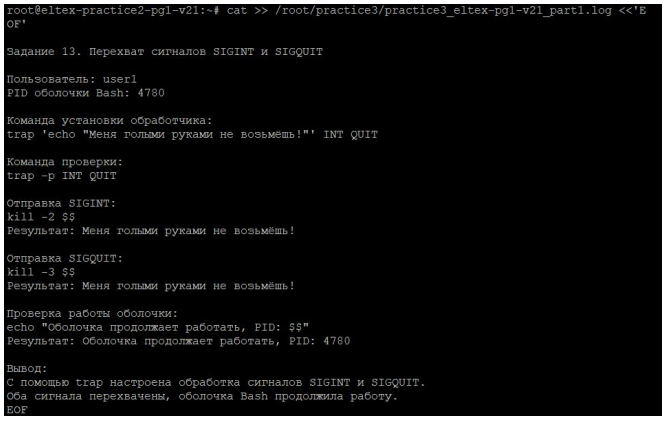


Рисунок 39 — Перехват SIGINT и SIGQUIT

Результат соответствующего этапа показан на рисунке 40.

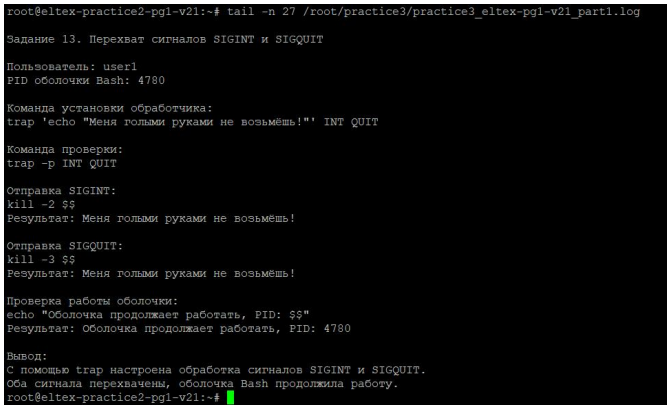


Рисунок 40 — Перехват SIGINT и SIGQUIT

Пункт 14. Создание дочерних процессов и наблюдение состояния Z.

Выполненные действия и команды:

```
(sleep 1 & exec /bin/sleep 30) &
ps -eo pid,ppid,stat,comm | awk 'NR==1 || $3 ~ /^Z/'
```

Результат и пояснение. Обнаружены зомби с PID/PPID: 4810/4809, 4815/4814, 4817/4816. Статус Z указывает на завершённого потомка, ожидающего получения статуса родителем.

Результат соответствующего этапа показан на рисунке 41.

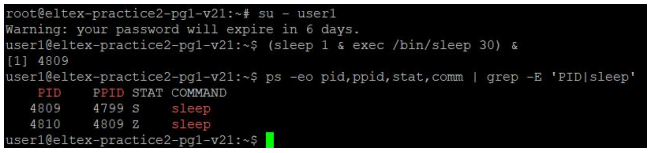


Рисунок 41 — Создание и обнаружение зомби-процессов

Результат соответствующего этапа показан на рисунке 42.

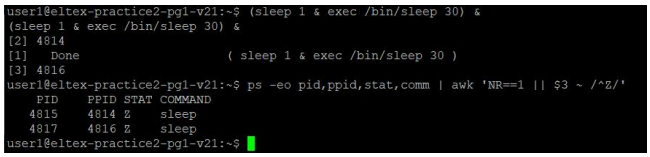


Рисунок 42 — Создание и обнаружение зомби-процессов

Результат соответствующего этапа показан на рисунке 43.

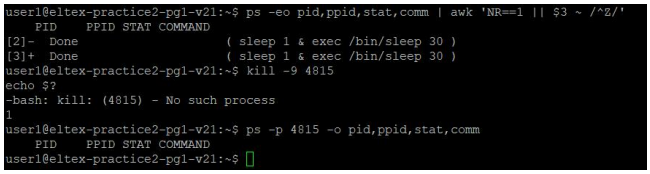


Рисунок 43 — Создание и обнаружение зомби-процессов

Пункт 15. Попытка досрочного завершения зомби и проверка результата.

Выполненные действия и команды:

```
kill -9 4815
echo $?
(sleep 1 & exec /bin/sleep 120) &
ps -eo pid,ppid,stat,comm | awk 'NR==1 || $3 ~ /^Z/'
kill -9 4825
echo $?
ps -p 4825 -o pid,ppid,stat,comm
```

Результат и пояснение. Первый процесс 4815 уже исчез (No such process, код 1). Повторный эксперимент: PID 4825, PPID 4824, STAT Z; kill -9 вернул код 0, но состояние Z сохранилось.

Особенности зомби-процессов (10 предложений). Зомби-процесс представляет собой уже завершившийся дочерний процесс, запись о котором ещё сохраняется в таблице процессов. В выводе ps такой процесс обозначается

буквой Z в столбце STAT. Он не выполняет пользовательский код и не расходует процессорное время. При этом ядро сохраняет PID и данные о завершении для родительского процесса. PPID позволяет установить, какому процессу принадлежит завершившийся потомок. Зомби возникает, когда родитель ещё не забрал статус потомка с помощью wait или аналогичного механизма. Команда kill -9 не может повторно завершить уже завершённый процесс. В опыте процесс 4825 сохранил статус Z после успешного вызова kill -9. Когда родитель забирает статус потомка, запись о зомби исчезает. Если родитель завершается раньше, потомок переподчиняется подходящему системному процессу, который может получить его статус.

Результат соответствующего этапа показан на рисунке 44.

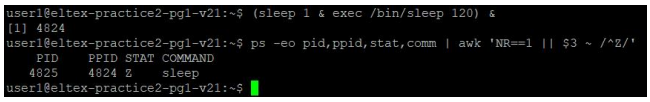


Рисунок 44 — Проверка невозможности завершить зомби сигналом SIGKILL

Результат соответствующего этапа показан на рисунке 45.

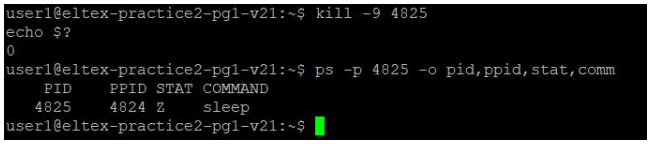


Рисунок 45 — Проверка невозможности завершить зомби сигналом SIGKILL

Результат соответствующего этапа показан на рисунке 46.

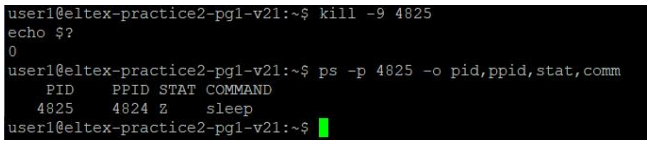


Рисунок 46 — Проверка невозможности завершить зомби сигналом SIGKILL

Результат соответствующего этапа показан на рисунке 47.

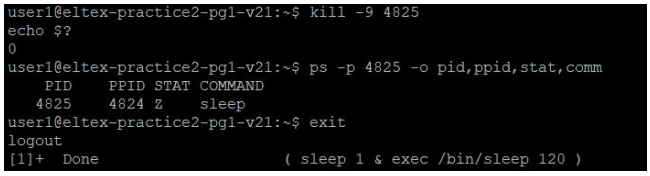


Рисунок 47 — Проверка невозможности завершить зомби сигналом SIGKILL

Результат соответствующего этапа показан на рисунке 48.

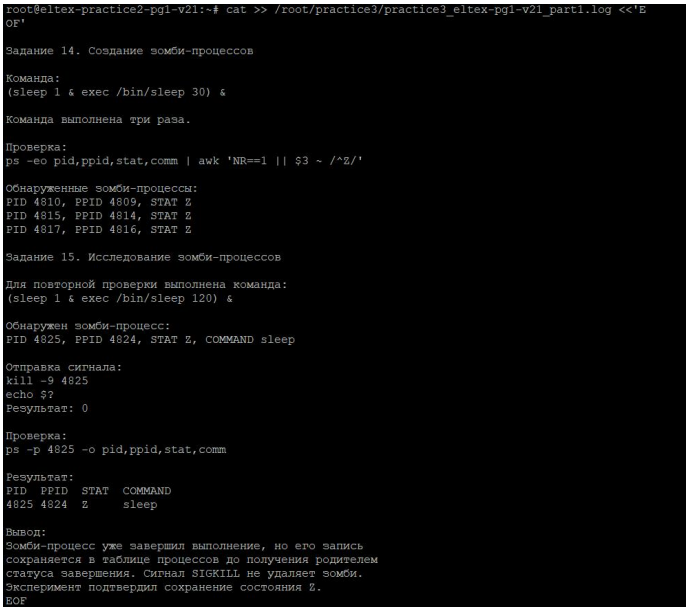


Рисунок 48 — Проверка невозможности завершить зомби сигналом SIGKILL

Результат соответствующего этапа показан на рисунке 49.

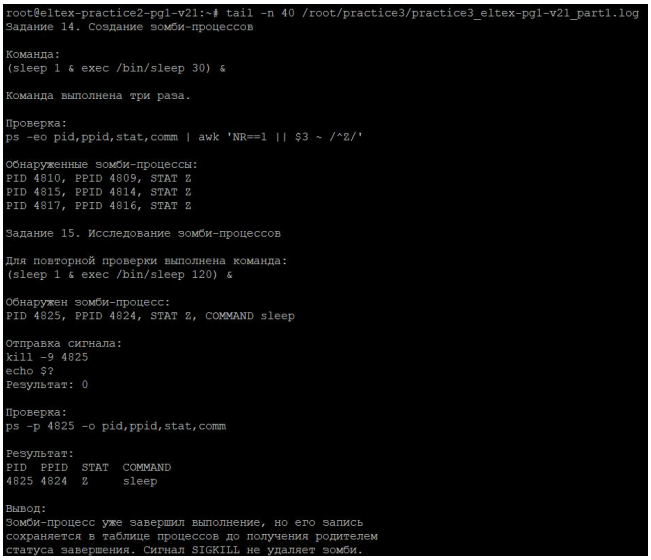


Рисунок 49 — Проверка невозможности завершить зомби сигналом SIGKILL

Вывод

В ходе практической работы освоены средства Linux для просмотра и анализа процессов, их потоков и использования памяти. С помощью ps и top исследованы показатели NLWP, VIRT, RES и SHR, а также различия реального и эффективного пользователей. На практике проверено действие сигналов SIGTERM, SIGINT, SIGQUIT и SIGKILL, настроен обработчик trap в Bash. Команды jobs, bg и fg позволили управлять приостановленными и фоновыми заданиями, а renice — изменить NICE процесса с 0 на 10. Эксперимент с дочерними процессами подтвердил появление состояния Z и невозможность удалить зомби отправкой SIGKILL. Результаты команд и скриншоты систематизированы по всем пятнадцати заданиям раздела.